



I would like to conduct an analysis in R in and QMD-file for the file: youtube_haitian_disinformation_video_meta.csv

The type of analysis I'd like to conduct consists of multiple layers and a preparation beforehand.

First the relevant packages that are needed later should be loaded. E.g. tidyverse, tidytext.

After this I'd like to view my dataset

After that I would like to remove the variables that are not relevant for the analysis I want to conduct. These variables are: date, tags, comments, transcript, transcript_str. However, if there is an easy way to count the comment_id variable from listing all the comments per video into a simple variable that just represents the comments as a count of comments per video that is preferred, as it would allow me to use the comment frequency for the correlation analysis I'd like to conduct later on. The comment_id variable can be found in youtube_haitian_disinformation_comment_reply_metadata.csv.

After these variables are removed, I'd like to create a new dichotomous variable: video_debunking, with 0 being "debunking" and 1 being "not debunking".

After this I will have to code the variable manually for each video with the corresponding value I'd like to assign.

Following

this I'd like to start my analysis. The goal is to find whether there is any

correlation between debunked or not debunked videos and interaction with the

type of videos. I think Pearson's R makes sense, but if there is another sort

of correlation analysis that makes more sense please let me know.

The

interaction sorts I'm interested in are views and likes. These variants are

also named views and likes respectively.

For the

analysis I'd like to create an exportable table which creates an overview per

video first. The table should include video title (var: title), views (var: views), and likes (var: likes).

After this the

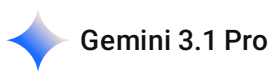
correlation analysis is the next step I'd like to take. An correlation table,

or perhaps another form of display that clearly displays relevant information

would be a nice thing to work towards.

Could you

help me write the syntax for this in R (environment R-studio)?



This sounds like a fascinating and highly relevant piece of research! Analyzing engagement metrics on disinformation and debunking videos can yield some incredibly valuable insights.

Your step-by-step approach is well-thought-out. I can absolutely help you write the `.qmd` (Quarto) syntax for this.

Before we dive into the code, I want to address your question about **Pearson's R**. Since your `video_debunking` variable is dichotomous (0 and 1) and your engagement metrics (views, likes, comments) are continuous, a standard Pearson's R mathematically becomes a **Point-Biserial correlation**. However, YouTube engagement metrics are notoriously heavily skewed (a few videos get millions of views, while most get very few). Because of this lack of a normal distribution, **Spearman's**

rank correlation is generally much safer and more accurate for this specific type of data. I will include the code for Spearman's in the syntax below.

Here is how you can set up your `.qmd` file. You can copy and paste these code chunks directly into your document.

1. Setup and Loading Packages

First, we load the required packages and read the data. I've added the `knitr` package to help with making your tables exportable and clean.

```
#| label: setup
#| warning: false
#| message: false

# Load necessary packages
library(tidyverse)
library(tidytext)
library(knitr)      # For creating nice, exportable tables
library(corr)       # For tidy correlation tables

# Read the datasets
# (Ensure these files are in your working directory)
video_meta <- read_csv("youtube_haitian_disinformation_video_meta.csv")
comment_meta <- read_csv("youtube_haitian_disinformation_comment_reply_metadata.csv")

# View the main dataset
glimpse(video_meta)
```

2. Data Preparation and Comment Counting

Next, we will count the comments per video from your secondary dataset, merge that count into your main dataset, drop the irrelevant columns, and create the empty `video_debunking` variable.

(Note: This assumes both datasets share a common column like `video_id`. If they use a different ID name, just change it in the `by =` argument below).

```
#| label: data-cleaning

# Count comments per video
comment_counts <- comment_meta %>%
  group_by(video_id) %>%
  summarise(comment_count = n(), .groups = 'drop')

# Merge, clean, and prep the dataset
video_prep <- video_meta %>%
  # Merge the comment counts
  left_join(comment_counts, by = "video_id") %>%
  # If a video had no comments in the comment file, replace NA with 0
  mutate(comment_count = replace_na(comment_count, 0)) %>%
  # Remove irrelevant variables
  select(-any_of(c("date", "tags", "comments", "transcript", "transcript_str"))) %>%
  # Create the dichotomous variable (filled with NA for now so you can code it)
  mutate(video_debunking = NA_real_)

# View the prepped data
head(video_prep)
```

3. The "Manual Coding" Intermission

Because you need to manually code the `video_debunking` variable, you have to pause the R script here. The easiest way to do this is to export your cleaned dataset, code it in Excel, and bring it back in.

Run this in your console (not necessarily in the final report):

```
write_csv(video_prep, "video_prep_for_coding.csv")
```

Once you have opened that CSV, typed `0` (debunking) or `1` (not debunking) for each video, and saved it, you can load it back into your Quarto document like this:

```
#| label: load-coded-data

# Read the manually coded file back in
# (Assuming you saved your coded file as 'video_prep_coded.csv')
video_final <- read_csv("video_prep_coded.csv")

# Ensure the new variable is treated as numeric for the correlation
video_final <- video_final %>%
  mutate(video_debunking = as.numeric(video_debunking))
```

4. Creating the Summary Table

Here, we select the specific variables you wanted and create a clean overview table using `knitr::kable`.

```
#| label: summary-table

# Create the overview table
overview_table <- video_final %>%
  select(title, views, likes, comment_count)

# Display the table cleanly
kable(overview_table,
      caption = "Overview of Video Engagement: Views, Likes, and Comments",
      format.args = list(big.mark = ",")) # Adds commas to large numbers
```

5. Correlation Analysis

Finally, we isolate your numeric variables and run the correlation matrix. We will use the `corr` package to create a very clean, readable table. I've set the method to `"spearman"` due to the skewed nature of YouTube data.

```
#| label: correlation-analysis

# Select only the numeric columns for correlation
correlation_data <- video_final %>%
  select(video_debunking, views, likes, comment_count) %>%
  # Remove any rows that might have missing data to ensure accurate stats
  drop_na()

# Calculate Spearman correlation
correlation_results <- correlation_data %>%
  correlate(method = "spearman", diagonal = 1)

# Display the correlation table
kable(correlation_results,
      digits = 3,
      caption = "Spearman Correlation Matrix: Debunking Status and Engagement")
```

This structure should give you exactly what you need for a clean, reproducible Quarto document.

How many videos are currently in your dataset? If it's a massive list, coding it manually one-by-one might take you days, and we could explore using `tidytext` to look for specific debunking keywords (like "false", "fake", "fact-check") to automatically pre-code some of them for you!

2026-05-09 13:47:57

You Asked



what does this do?

```
#| label: setup
```

```
#| warning: false  
#| message: false
```

Gemini 3.1 Pro

These lines are **chunk options** specific to Quarto (`.qmd`) files. They tell the rendering engine how to handle and format the output of the R code block they are attached to. The `#|` (hash-pipe) syntax is the standard way to pass these instructions to Quarto.

Here is exactly what each line does:

- **`#| label: setup`** : This gives your code chunk a specific name (in this case, "setup"). Naming your chunks is highly recommended because if your code ever throws an error while rendering the final document, the error log will tell you exactly which chunk failed (e.g., "Error in chunk 'setup'"). It also helps if you want to cross-reference or cache specific parts of your code later.
- **`#| warning: false`** : When you load packages or run certain functions in R, R will often print warning messages (for example, "Warning: package 'tidyverse' was built under R version 4.3.2"). Setting this to `false` prevents those warnings from showing up in your final exported document, keeping it looking clean and professional.
- **`#| message: false`** : Similar to warnings, R often prints generic informational messages when executing code. For example, when you load `library(tidyverse)` , R spits out a block of text telling you which specific packages were attached and if there are any function conflicts. Setting this to `false` hides all of that backend chatter from your final rendered report.

In short, using `warning: false` and `message: false` in your "setup" chunk ensures that your final document only shows the beautiful tables and analyses you actually want to present, rather than a wall of technical R text.

2026-05-09 15:09:33 You Asked 

I'm having some trouble with the manual coding in step three. I'd rather assign the values in R. How do I do that?

Gemini 3.1 Pro

It is completely understandable that you'd want to keep everything contained within R. Constantly exporting and importing files breaks your workflow and can introduce errors.

If you want to assign these values directly in R, the best method depends on **how** you are deciding which video is a debunking video (0) and which is not (1). Here are the two most effective ways to do this in your `.qmd` script.

Method 1: Using a List of Specific Video IDs (Manual but in R)

If you have looked at your list of videos and already know exactly which ones are the debunking videos, you can create a list of their unique `video_id` s. Then, R will assign a `0` to any video on that list, and a `1` to everything else.

```
#| label: assign-by-id

# 1. Type the exact video IDs of your debunking videos here
debunking_videos <- c("ID_number_1", "ID_number_2", "ID_number_3")

# 2. Assign the 0 or 1 based on that list
video_final <- video_prep %>%
  mutate(video_debunking = ifelse(video_id %in% debunking_videos, 0, 1))

# View the result to ensure it worked
table(video_final$video_debunking)
```

Note: Make sure to replace "ID_number_1", etc., with the actual IDs from your dataset, wrapped in quotation marks.

Method 2: Keyword Searching (Automated)

If you haven't identified the specific videos yet, you can use the `stringr` package (which is loaded automatically with `tidyverse`) to search the video titles. If the title contains words like "fake," "debunk," or "false," R assigns it a `0`. Otherwise, it gets a `1`.

This uses the `case_when()` function, which is excellent for conditionally assigning values.

```
#| label: assign-by-keyword

video_final <- video_prep %>%
  mutate(video_debunking = case_when(
    # Search the 'title' column for specific keywords (converted to lowercase to catch everything)
    str_detect(str_to_lower(title), "debunk|fake|false|fact-check|hoax") ~ 0,

    # If the keywords aren't found, assign 1
    TRUE ~ 1
  ))

# View the result
table(video_final$video_debunking)
```

Which one to choose?

- Use **Method 1** if you need 100% precision and have manually watched/reviewed the videos to verify their exact content.

- Use **Method 2** if your dataset is massive and you are comfortable using the title text as a reliable proxy for the video's content.

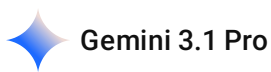
Once you run either of these code blocks, your `video_final` dataset will have the `video_debunking` variable fully populated, and you can proceed directly to step 4 (Creating the Summary Table) from the previous instructions without ever leaving RStudio!

2026-05-09 15:11:04

You Asked



I'd like to make `video_debunking` a factor variable 0 = debunking, 1 = not debunking and assign the values manually per video



Making `video_debunking` a factor is an excellent practice in R, especially when you are building tables or plots later, as it will display the clear labels ("debunking") instead of just numbers.

However, there is one important **catch**: correlation functions in R (like the `correlate()` function we are using later) require strictly **numeric** data. If you change your variable into a factor, the correlation matrix will drop it and throw an error.

To get the best of both worlds, we will assign the values manually by `video_id`, create your factor variable for tables, and keep a numeric version strictly for the correlation.

Here is the exact syntax to do this in your `.qmd` file:

Assigning Values and Creating the Factor

To manually code this in the script without breaking if the row order accidentally changes, the safest method is to list the exact `video_id` s of the videos that are debunking (0). Everything else will default to not debunking (1).


```

#| label: manual-coding-and-factors

# 1. Manually list the exact video IDs that are DEBUNKING
# (Replace these placeholders with your actual video IDs)
debunking_videos <- c("ID_1", "ID_2", "ID_3", "ID_4")

# 2. Apply the manual coding and create the factor
video_final <- video_prep %>%
  mutate(
    # First, create the numeric version for your correlation analysis
    video_debunking_num = ifelse(video_id %in% debunking_videos, 0, 1),

    # Next, create the factor version for your tables and summaries
    video_debunking_factor = factor(video_debunking_num,
                                    levels = c(0, 1),
                                    labels = c("debunking", "not debunking"))
  )

# View a quick summary to make sure your factor applied correctly
summary(video_final$video_debunking_factor)

```

Updating the Next Steps

Because we changed the variable names to be very specific, you just need to make tiny adjustments to your subsequent code chunks so they use the correct version of the variable.

For your Summary Table (Step 4), use the Factor:

```

#| label: summary-table

# Use the factor version so the table reads "debunking" / "not debunking"
overview_table <- video_final %>%
  select(title, video_debunking_factor, views, likes, comment_count)

kable(overview_table,
      caption = "Overview of Video Engagement",
      format.args = list(big.mark = ","))

```

For your Correlation Analysis (Step 5), use the Numeric:

```

#| label: correlation-analysis

# Use the numeric version so the correlation math works
correlation_data <- video_final %>%
  select(video_debunking_num, views, likes, comment_count) %>%
  drop_na()

correlation_results <- correlation_data %>%
  correlate(method = "spearman", diagonal = 1)

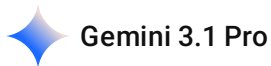
kable(correlation_results,
      digits = 3,
      caption = "Spearman Correlation Matrix")

```



image_920758.png

Can you help me interpret this? i'm not familiar with what certain values of Spearman mean



Gemini 3.1 Pro

It is completely normal to need a translation for this! Staring at a grid of decimals can be unintuitive if you don't look at correlation matrices every day.

Here is a straightforward guide to reading your Spearman Correlation Matrix, starting with what the numbers mean and then breaking down your specific results.

The Cheat Sheet for Spearman Values

Spearman correlation coefficients always fall between **-1.0** and **1.0**.

- **The Sign (+ or -)** tells you the *direction* of the relationship.
 - A **positive** number means as one variable goes up, the other goes up.
 - A **negative** number means as one variable goes up, the other goes *down*.
- **The Number** tells you the *strength* of the relationship. A common rule of thumb in social sciences is:
 - **0.00 to 0.29**: Weak relationship
 - **0.30 to 0.49**: Moderate relationship
 - **0.50 to 0.69**: Strong relationship
 - **0.70 to 1.00**: Very strong relationship
- (Note: The diagonal line of **1.000** just represents a variable perfectly correlating with itself, which is why it forms a straight line down the matrix).

Interpreting Your Specific Results

1. The Big Finding: Debunking vs. Engagement Remember how you coded this variable: **0 = debunking** and **1 = not debunking**. Because "not debunking" is the higher number mathematically (1 is greater than 0), a *positive* correlation means that the "not debunking" videos are associated with higher engagement.

- **video_debunking_num vs. likes (0.551)**: This is a **strong positive** correlation. It suggests that "not debunking" videos (disinformation) receive significantly more likes than the debunking videos.
- **video_debunking_num vs. views (0.524)**: This is also a **strong positive** correlation. Similarly, the "not debunking" videos are getting substantially more views than the debunking videos.
- **video_debunking_num vs. comment_count (0.215)**: This is a **weak positive** correlation. While "not debunking" videos do get slightly more comments, the relationship isn't nearly as strong as it is for views or likes.

2. The Expected Technical Findings: How Engagement Relates to Itself

- **views vs. likes (0.941)**: This is a **very strong positive** relationship. This is entirely expected on YouTube; videos that get lots of views inherently get lots of likes.
- **views vs. comment_count (0.534)** and **likes vs. comment_count (0.462)**: These are **moderate to strong positive** relationships. While more views and likes generally mean more comments, it's not a near-perfect 1:1 ratio. People are much more likely to passively watch or hit the like button than they are to take the time to write a comment.

The Main Takeaway for Your Analysis: Based on this matrix, your data strongly suggests that disinformation ("not debunking") videos are heavily outperforming debunking videos in terms of pure reach (views) and passive approval (likes), though they don't trigger drastically more active conversation (comments).

2026-05-09 15:43:03 You Asked 

So if the debunking videos had higher interaction the Spearman's correlation for views, likes and comments would have been negative, despite debunking having a value of 0? Assuming debunking videos had higher interaction, wouldn't it have been more logical for the correlation values to be somewhere around 0 since, debunking = 0 and $0 * \text{something else} = 0$?

 Gemini 3.1 Pro

You are absolutely right on the first point: if the debunking videos (0) had higher interaction than the not-debunking videos (1), the correlation would indeed be negative! That shows you have a great grasp of how the direction (+ or -) works.

However, the second part of your thought is a very common mathematical misconception. You are thinking of the math behind correlation as simple multiplication ($0 \times \text{views} = 0$), but correlation doesn't work that way. Correlation measures the **relationship or pattern** between variables, not the raw product of their numbers.

Here is why it wouldn't equal 0, and how the math actually treats your 0 and 1:

1. A Correlation of 0 Means "No Relationship"

In statistics, a correlation coefficient of 0.00 means **there is absolutely no pattern**.

If your correlation had been 0, it wouldn't mean the debunking videos got zero views. It would mean that knowing whether a video is debunking (0) or not (1) tells you *nothing* about its views. The views would be completely random and roughly equal across both categories.

2. The 0 and 1 are Just "Steps" (Dummy Variables)

When you code a variable as 0 and 1, the correlation formula isn't multiplying the views by zero. Instead, it treats 0 and 1 as sequential steps on a ladder:

- **Step 0:** Debunking
- **Step 1:** Not Debunking

The correlation formula simply asks: *"When I step UP the ladder from 0 to 1, what happens to the views?"*

- **If views go UP** when you move from 0 to 1 (meaning the "1" videos have more views) → **Positive correlation** (This is what happened in your data).
- **If views go DOWN** when you move from 0 to 1 (meaning the "0" videos had more views) → **Negative correlation**.
- **If views don't change** in a predictable way between 0 and 1 → **0 correlation**.

Because 0 and 1 are just acting as placeholder labels to show two different groups, you could have coded Debunking as 98 and Not Debunking as 99, and you would have gotten the exact same correlation values in your matrix!

2026-05-09 15:47:55 You Asked 

could you explain the formula in depth to me?

I would be happy to break the math down for you! The secret to understanding why your 0s and 1s don't destroy the math is understanding what **Spearman's rank correlation** actually does behind the scenes.

The most important thing to know about Spearman's formula is that **it doesn't care about your raw numbers**. Before it does any math, it translates all your data into **ranks** (like finishing places in a race).

Here is an in-depth look at how the formula works and what it does to your data.

Step 1: The Ranking Transformation

Imagine you have three videos with the following views: 50 views, 100 views, and 2,000,000 views. Spearman completely throws away those raw numbers and assigns them ranks from lowest to highest:

- 50 views → Rank 1
- 100 views → Rank 2
- 2,000,000 views → Rank 3

It does the exact same thing to your `video_debunking` variable. All the videos coded as 0 are tied for lowest, so they get the lower ranks. All the videos coded as 1 are tied for highest, so they get the higher ranks. R assigns an "average rank" to ties so the math stays balanced. **This is why the 0 doesn't act like a mathematical black hole—it is no longer a 0, it is just "Rank 1".**

Step 2: The Formula

Once everything is converted into a rank (1st, 2nd, 3rd, etc.), Spearman uses a formula to compare the ranks of one variable against the ranks of the other.

The classic textbook formula for Spearman's rank correlation coefficient (r_s) looks like this:

$$r_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

Step 3: Breaking Down the Formula

Here is exactly what each piece of that equation means in plain English:

- r_s : This is the final Spearman correlation coefficient (the numbers like 0.524 or 0.215 that you saw in your matrix).
- 1—: The formula starts at a perfect 1.0 (a perfect positive correlation) and subtracts from it based on how "out of order" the ranks are.
- d_i (**The Difference**): For every single video, the formula takes its rank in Variable A (e.g., Debunking) and subtracts its rank in Variable B (e.g., Views). If a video was Rank 1 for Debunking and Rank 1 for Views, the difference (d) is 0.

- d_i^2 (**Squaring the Difference**): The difference is multiplied by itself to ensure all negative numbers become positive, and to heavily penalize videos that have wildly different ranks (e.g., Rank 1 in Debunking but Rank 100 in Views).
- $\sum$ (**Sigma / Sum**): This symbol just means "add them all up." You add up the squared differences for every single video in your dataset.
- 6: This is a mathematical constant used to scale the differences appropriately. (It's always 6, no matter your data).
- n : The total number of videos in your dataset. If you have 500 videos, $n = 500$. The denominator $n(n^2 - 1)$ figures out the maximum possible "messiness" based on the size of your dataset, ensuring the final number always stays between -1.0 and 1.0.

The Logic in Action

If your ranks perfectly align—meaning the lowest-ranked debunking video also has the lowest-ranked views, the second-lowest debunking video has the second-lowest views, and so on—the difference (d) for every single video is 0.

If you plug a 0 into the top half of that fraction, the whole fraction becomes 0. $1 - 0 = 1.0$ (A perfect positive correlation!)

Conversely, the more "out of sync" the ranks are, the bigger the fraction becomes, and the further away from 1.0 the final score falls.

Does this ranking concept make it clearer why your categorical 0s and 1s play nicely with your continuous view counts?

2026-05-10 16:38:48 You Asked 

 image_eb4311.png

why does this take so long when I have just asked to open tidyverse from the library

 Gemini 3.1 Pro

Seeing that green bar hang forever just to load a single package is incredibly frustrating! Loading `tidyverse` should usually only take a few seconds.

When RStudio gets stuck on a simple `library()` command, it is almost always due to one of these four reasons. Here is what is likely happening and how to fix it:

1. R is waiting for you to answer a hidden question (Most Common)

This is the #1 culprit. Sometimes when you load or install a package, R encounters a warning or needs permission (e.g., asking if you want to update older packages, or asking you to pick a download mirror).

- **The Fix:** Look down at your **Console pane** (usually the bottom-left window in RStudio). If you see a prompt asking you a Yes/No question, or a blinking cursor waiting for you to type `Y`, `N`, or a number, type your answer and hit Enter. The chunk will immediately finish running.

2. You clicked "Run All" and it's stuck on a previous step

The progress bar says "Run Chunks" (plural). If you clicked the "Run All" button or "Run All Above", RStudio executes them sequentially. It might not actually be stuck on `library(tidyverse)`; it might still be trying to read your `youtube_haitian_disinformation_video_meta.csv` file if it's exceptionally large, or it's stuck on a previous chunk.

- **The Fix:** Click that red **Stop** button in your screenshot. Then, go to the specific chunk with `library(tidyverse)` and click the small green "Play" button *just for that specific chunk*.

3. It's loading 8 packages at once

`tidyverse` isn't actually a single package; it is a "wrapper" that loads a massive ecosystem of packages all at once (including `ggplot2`, `dplyr`, `tidyr`, `readr`, `purrr`, `tibble`, `stringr`, and `forcats`). If you are working on an older computer, a heavily restricted work laptop, or saving your files to a cloud/network drive (like OneDrive or a university server), fetching all those files can genuinely take a minute the first time you do it in a new session.

4. Your R Session has "Ghosted"

Sometimes the R engine simply freezes in the background, but the RStudio interface keeps running, leaving you staring at an infinite loading bar.

- **The Fix:** Click the red **Stop** button. Go to the top menu bar, click **Session**, and select **Restart R** (or press `Ctrl + Shift + F10` / `Cmd + Shift + F10`). Once the console is clear, try running the chunk again.

2026-05-10 16:42:57 You Asked 

none of these options helped

That is incredibly frustrating. If the standard fixes didn't clear the jam, it means we are likely dealing with a deeper issue—either a corrupted workspace, a broken dependency, or a conflict with Quarto itself.

Let's try the next-level diagnostic steps to force R to tell us exactly what is breaking.

1. The Console Test (Isolating the Issue)

We need to figure out if Quarto (the `.qmd` file system) is freezing, or if R itself is broken.

- Click the red **Stop** button to cancel the hanging chunk.
- Go down to your **Console** pane (the tab that usually says "Console" next to "Terminal" at the bottom).
- Type `library(tidyverse)` directly into the console and hit **Enter**.
- *If it loads instantly:* Your Quarto background engine is hanging.
- *If it freezes there too:* The `tidyverse` installation or your R environment is the problem.

2. Clear the Hidden Workspace (The Silent Killer)

By default, RStudio often tries to save and reload everything you've ever worked on in a hidden file called `.RData`. If a previous session corrupted a variable, loading a package can trigger a massive freeze.

- Go to the top menu: **Session** → **Clear Workspace...** (Confirm if it asks).
- Then go to **Tools** → **Global Options**.
- Under the "General" tab, look for "**Restore .RData into workspace at startup**" and **uncheck** it.
- Set "Save workspace to `.RData` on exit:" to **Never**.
- Restart R (`Ctrl + Shift + F10` / `Cmd + Shift + F10`) and try again.

3. The Dependency Check

Because `tidyverse` loads about 8 different packages, one of them might be broken, but the main wrapper isn't showing you the error.

- In the console, try loading them one by one:
 - Type `library(dplyr)` and hit Enter.
 - Type `library(ggplot2)` and hit Enter.
 - Type `library(stringr)` and hit Enter.
- If one of these freezes or throws a big red error (like "package 'rlang' was built under R version..."), we have found our culprit.

4. The "Nuke and Pave" Option

If all else fails, the package library might be corrupted and needs a fresh install.

- Run this command in your console to force a fresh download of the package and all its underlying requirements: `install.packages("tidyverse", dependencies = TRUE)`
- Let it run (it might take a few minutes), restart R, and try your code chunk again.

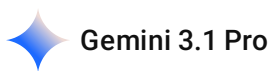
When you try typing `library(tidyverse)` directly into the bottom console (Step 1), what exactly happens—does it freeze there too, or does it work normally?

2026-05-10 16:43:47

You Asked



this is for the rmd file



Ah, got it! That is a very helpful clarification. R Markdown (`.rmd`) and Quarto (`.qmd`) are very similar, but `.rmd` files have a few specific quirks when it comes to the background engine (called `knitr`) that can cause these infinite loading bars.

The previous steps (like testing it in the console or clearing your workspace) still apply perfectly to `.rmd` files, but since we know it's specifically an R Markdown file, here are the two most likely `.rmd` - specific culprits and how to fix them:

1. Clear the Knitr Cache (The most likely R Markdown fix)

R Markdown tries to be helpful by saving (caching) the results of your code chunks so it doesn't have to re-run them every time. If that cache file gets corrupted, the chunk will just spin forever trying to read it.

- Look at the top of your script window where the **Knit** button is (it has a little ball of yarn next to it).
- Click the small **gear icon** next to the Knit button.
- Select **Clear Knitr Cache...** from the dropdown menu.
- Restart R (`Ctrl + Shift + F10` or `Cmd + Shift + F10`) and try running the chunk again.

2. Reset the RStudio Project State

If you are working inside an R Project (a `.Rproj` file), RStudio saves your window layouts and open files in a hidden folder. If this gets corrupted, R Markdown files specifically tend to freeze up when running chunks inline.

- **Save** your script and close RStudio completely.

- Open your computer's normal file explorer (Finder on Mac, File Explorer on Windows) and navigate to the folder where your project is saved.
- Look for a hidden folder called `.Rproj.user`. (You may need to enable "Show hidden files" in your computer's view settings to see it).
- **Delete** that `.Rproj.user` folder. (Don't worry, this is perfectly safe—it will just reset your open tabs, it won't delete your code or data).
- Reopen your project and try again.

3. A Quick Syntax Check

Since you are using an `.rmd` file, I want to double-check one small detail. In my previous examples, I gave you Quarto syntax for the chunk options (the lines starting with `#|`). While newer versions of R Markdown can read this just fine, older versions sometimes get confused by it. To be completely safe, you can change your setup chunk to the classic R Markdown format to see if that stops the freezing:

```
r

```{r setup, warning=FALSE, message=FALSE}
library(tidyverse)
```

If you try running ``library(tidyverse)`` completely outside of the `.rmd`` file directly in the bottom **Console**, does it still freeze there, or does it load successfully?

2026-05-10 16:47:51 You Asked 

I got the following error, while I already have tinytex installed when trying to knit a pdf:

```
"C:/Program
Files/RStudio/resources/app/bin/quarto/bin/tools/pandoc" +RTS -
K512m -RTS DH-correlation-analysis-RMD.knit.md -to latex -from
markdown+autolink_bare_uris+tex_math_single_backslash -output
DH-correlation-analysis-RMD.tex -lua-filter
"C:\Users\caspe\AppData\Local\R\win-
library\4.4\rmarkdown\rmarkdown\lua\pagebreak.lua" -lua-filter
"C:\Users\caspe\AppData\Local\R\win-
library\4.4\rmarkdown\rmarkdown\lua\latex-div.lua" -embed-
resources -standalone -highlight-style tango -pdf-engine pdflatex -
variable graphics -variable "geometry:margin=1in" output file: DH-
correlation-analysis-RMD.knit.md
```

[WARNING] Deprecated: `--highlight-style`. Use `--syntax-highlighting` instead.

Error: LaTeX failed to compile DH-correlation-analysis-RMD.tex. See <https://yihui.org/tinytex/r/#debugging> for debugging tips.

In addition: Warning messages:

1: In `attr(x, "match.length")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

2: In `attr(x, "match.length")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

3: In `attr(x, "match.length")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

4: In `attr(x, "match.length")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

5: In `attr(x, "match.length")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

6: In `attr(x, "match.length")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

7: In `attr(x, "knit_meta")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

8: In `attr(x, "knit_meta")` : '`xfun::attr()`' is deprecated.

Use '`xfun::attr2()`' instead.

See `help("Deprecated")`

9: In `system2(..., stdout = if (use_file_stdout()) f1 else FALSE, stderr = f2)` :

"`pdflatex`" not found

Execution halted

No LaTeX installation detected (LaTeX is required to create PDF output). You should install a LaTeX distribution for your platform: <https://www.latex-project.org/get/>

If you are not sure, you may install TinyTeX in R:

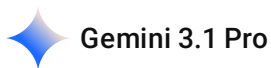
```
tinytex::install_tinytex()
```

Otherwise consider MiKTeX on Windows - <http://miktex.org>

MacTeX on macOS - <https://tug.org/mactex/>

(NOTE: Download with Safari rather than Chrome \_strongly\_ recommended)

Linux: Use system package manager



This is a classic and very frustrating RStudio trap! It is completely understandable why you are confused, especially since you know you already installed `tinytex` .

The core of the problem is right at the bottom of your error log: `"pdflatex" not found` and `No LaTeX installation detected` .

Here is exactly why this happens and how to fix it:

## The "TinyTeX" Trap

In R, installing the **R package** named `tinytex` (which you likely did via `install.packages("tinytex")` ) does **not** actually install the LaTeX software on your computer. It only installs the R *helper functions* that know how to talk to LaTeX.

Because RStudio can't find the actual LaTeX software engine (called `pdflatex` ) installed on your Windows machine, the PDF creation process crashes.

## How to Fix It

To actually install the underlying LaTeX software distribution that RStudio needs, you need to use one of those helper functions you downloaded.

**Step 1:** Go down to your **Console** pane (not your script). **Step 2:** Copy and paste this exact command and hit Enter:

```
r

tinytex::install_tinytex()
```

**Step 3:** Let it run. It will download several files from the internet and install the actual LaTeX engine to your computer. It might take a few minutes. **Step 4:** Once it finishes, completely close RStudio and

reopen it. (This ensures your computer registers that the new software is installed). **Step 5:** Try clicking **Knit to PDF** again.

*(Note: Don't worry about those warning messages regarding `xfun::attr()` . That is just harmless background noise caused by minor version differences between R packages and won't stop your PDF from building once LaTeX is installed.)*

Were you able to run the `install_tinytex()` command successfully in the console, or did it throw any new errors?

2026-05-10 20:13:39 You Asked 

Could you make this into a table in a png view?:

Overview of Video		
Engagement	title	video_debunking_factorviewslikescomment_count
Why Trump keeps pushing debunked claim about Haitian immigrants	debunking	312,2224,3275,132
Springfield mayor responds to JD Vance's claims about Haitian immigrants	not debunking	320,8345,9384,070
Trump, Harris face off at the ABC News Presidential Debate	not debunking	1,858,43627,85610,803
Trump amplifies false claims about Haitian immigrants eating pets	debunking	43,414546820
Springfield, Ohio, man reported Haitian migrants snatching geese out of park just 2 weeks ago	not debunking	236,9245,3631,670
Springfield resident describes impact of Haitian migrants on community	not debunking	1,342,40535,9199,085
Springfield neighbors describe immigrant situation; say they believe pet-snatching story	not debunking	261,9904,5753,581
JD Vance exposed for knowingly lying about 'pet-eating' rumors in Springfield, OH	debunking	280,0725,8301,917
'You think this is the last cup of B.S.': Steele slams Trump campaign for lying about migrants	debunking	442,4178,7612,108
Ohio residents rage at officials over Haitian migrant surge: 'No vacancy'	not debunking	1,233,28734,7789,843
Trump spreads more lies about Springfield, Ohio	debunking	297,2493,3704,536
Springfield, Ohio resident: This is a dystopian nightmare	not debunking	453,85310,533144
Haitian community in Springfield, Ohio, under threat after debunked		

commentsdebunking36,872459916Reporter Presses JD Vance About 'Debunked False Claims' About Haitians In Springfield, Ohio  
debunking82,7682,238926Criminal charges filed by Haitian nonprofit against Trump and JD Vance  
debunking44,9191,4961,290'NO DOGS OR CATS' Haitian immigrant pleads with Trump and Springfield, Ohio residents  
debunking54,2228091,349Missing cat at center of Springfield conspiracy theory found hiding in basement  
debunking332,4735,6312,270Trump falsely claims Haitian migrants are eating pets in Ohio  
debunking88,2419861,974JD Vance claims on Ohio's Haitian immigrants eating pets debunked  
debunking86,4826081,368Dana Bash and JD Vance clash over baseless claims about Haitian immigrants (Part 1/2)  
debunking838,58912,1641,536JD Vance reacts to Springfield father saying he politicized his son's death (Part 2/2)  
not debunking379,1626,603101JD Vance defends amplifying false claims about immigrants  
debunking195,8463,4673,969Residents in Springfield, Ohio react to accusations of immigrants abducting and eating pets  
not debunking243,8551,2961,617



I would love to just hand you a perfectly formatted PNG file right now, but I need to be completely candid about how my image generation works. My image tool (which is highly advanced for artistic or photographic generation) genuinely struggles with rendering long strings of exact text and numbers. If I tried to generate an image of this table, the text would come out looking garbled and completely illegible.

However, since you are already working in RStudio, the absolute best way to get a crisp, professional PNG of this exact table is to have R generate and save it for you!

First, here is the clean Markdown version of your data so you have it easily accessible:

title	video_debunking_factor	views	likes	comment_count
Why Trump keeps pushing debunked claim about Haitian immigrants	debunking	312,222	4,327	5,132

title	video_debunking_factor	views	likes	comment_count
Springfield mayor responds to JD Vance's claims about Haitian immigrants	not debunking	320,834	5,938	4,070
Trump, Harris face off at the ABC News Presidential Debate	not debunking	1,858,436	27,856	10,803
Trump amplifies false claims about Haitian immigrants eating pets	debunking	43,414	546	820
Springfield, Ohio, man reported Haitian migrants snatching geese out of park just 2 weeks ago	not debunking	236,924	5,363	1,670
Springfield resident describes impact of Haitian migrants on community	not debunking	1,342,405	35,919	9,085
Springfield neighbors describe immigrant situation; say they believe pet-snatching story	not debunking	261,990	4,575	3,581
JD Vance exposed for knowingly lying about 'pet-eating' rumors in Springfield, OH	debunking	280,072	5,830	1,917
'You think this is the last cup of B.S.?': Steele slams Trump campaign for lying about migrants	debunking	442,417	8,761	2,108
Ohio residents rage at officials over Haitian migrant surge: 'No vacancy'	not debunking	1,233,287	34,778	9,843
Trump spreads more lies about Springfield, Ohio	debunking	297,249	3,370	4,536
Springfield, Ohio resident: This is a dystopian nightmare	not debunking	453,853	10,533	144
Haitian community in Springfield, Ohio, under threat after debunked comments	debunking	36,872	459	916
Reporter Presses JD Vance About 'Debunked False Claims' About Haitians In Springfield, Ohio	debunking	82,768	2,238	926
Criminal charges filed by Haitian nonprofit against Trump and JD Vance	debunking	44,919	1,496	1,290
'NO DOGS OR CATS' Haitian immigrant pleads with Trump and Springfield, Ohio residents	debunking	54,222	809	1,349

title	video_debunking_factor	views	likes	comment_count
Missing cat at center of Springfield conspiracy theory found hiding in basement	debunking	332,473	5,631	2,270
Trump falsely claims Haitian migrants are eating pets in Ohio	debunking	88,241	986	1,974
JD Vance claims on Ohio's Haitian immigrants eating pets debunked	debunking	86,482	608	1,368
Dana Bash and JD Vance clash over baseless claims about Haitian immigrants (Part 1/2)	debunking	838,589	12,164	1,536
JD Vance reacts to Springfield father saying he politicized his son's death (Part 2/2)	not debunking	379,162	6,603	101
JD Vance defends amplifying false claims about immigrants	debunking	195,846	3,467	3,969
Residents in Springfield, Ohio react to accusations of immigrants abducting and eating pets	not debunking	243,855	1,296	1,617

## How to Create the PNG in RStudio

To turn `overview_table` directly into a beautiful PNG image file saved right to your computer, you can use the `gt` package (Grammar of Tables).

### 1. Install the necessary packages (Run this in your console once):

```
r
install.packages(c("gt", "webshot2"))
```

### 2. Run this chunk in your script to generate and save the image:

```
r
#| label: save-table-as-png
#| warning: false
#| message: false

library(gt)

Create a beautiful, formatted table
png_table <- overview_table %>%
```



```
gt() %>%
 tab_header(
 title = "Overview of Video Engagement"
) %>%
 cols_label(
 title = "Title",
 video_debunking_factor = "Debunking Status",
 views = "Views",
 likes = "Likes",
 comment_count = "Comments"
) %>%
 fmt_number(
 columns = c(views, likes, comment_count),
 decimals = 0
) %>%
 opt_row_striping() # Adds nice alternating row colors

Save it as a PNG directly to your working directory
gtsave(png_table, "video_engagement_overview.png")
```

Once you run that, you will find a crisp, perfectly legible `video_engagement_overview.png` file waiting in the same folder where your project is saved!